

A WebAssembly Port of Grand Theft Auto III

UCS503

Submitted to: Professor Nisha Thakur

Thapar Institute of Engineering and Technology

Antriksh (1024031034) Divyam Jain (1024031032)

1 Higher-order goal

This project ports the Grand Theft Auto III engine to the browser via WebAssembly (WASM), so it runs directly inside a standard web browser without any native installation step. The engine itself is **re3**, an existing, publicly available open-source C++ reimplementation of GTA III built on documented reverse-engineering research. We compile re3's own C++ source directly to WebAssembly with Emscripten, rather than rewriting the engine in another language -- our contribution is the browser porting layer: rendering, audio, input, and persistent storage bindings, plus a thin HTML/JS shell around the compiled module.

2 Time-to-value

- We deliver meaningful increments quickly by getting the engine to boot in-browser and reach an interactive state within a short iteration window, before chasing full feature parity.
- We prioritize fast feedback over perfection by using CMake/Emscripten builds that also still produce an ordinary native Windows/Linux binary from the same source, so behavior can be diffed against native at any time.
- Success is defined by what can be run and measured in the browser in the first iteration (boot to an interactive state), then improved based on profiling and playtesting.

3 Problem Statement

Grand Theft Auto III, like most software of its era, was written against platform APIs such as native windowing, OpenGL/Direct3D, local-disk file I/O, and native threading. None of these have a direct browser equivalent, so the engine cannot run in a web browser without a porting layer.

We use re3's existing C++ engine source directly -- an already-mature, actively maintained, publicly available reimplementation of GTA III's file formats and game logic -- rather than writing a new engine from scratch. Porting it to the browser requires closing three technology gaps:

- **Rendering:** there is no native OpenGL/Direct3D in a browser; re3's own rendering library (librw, a RenderWare-compatible renderer) is compiled through Emscripten to target WebGL2.
- **Audio:** native audio backends have no browser equivalent; re3's existing OpenAL-based audio code is compiled through Emscripten's OpenAL-to-Web-Audio implementation, unmodified.

- **Filesystem and concurrency:** the original engine assumes local-disk access and native threads, while browsers sandbox both behind browser-native storage (IndexedDB) and a restricted threading model. In practice, the engine's one background thread (CD streaming) did not need real concurrency at all once ported, and now runs synchronously on the main thread instead.

Why this matters now (contextual relevance): a working browser build removes installation friction entirely, makes the engine trivially shareable for study, demonstration, or preservation, and is a concrete, well-scoped systems-porting exercise that exercises real toolchain, performance, and platform-compatibility engineering.

4 Proposed Solution

4.1 Overview

Build a browser-playable WebAssembly build of the GTA III engine with the following components:

- **Language:** the engine and all game logic are re3's existing, unmodified C++ source; the browser side is a static HTML page, a small amount of CSS, and a thin vanilla-JS shell (no framework, no build step of its own).
- **Build pipeline:** Emscripten (emcc/em++) compiles the C++ to a .wasm module plus a JS glue/loader file, driven by CMake -- the same CMakeLists.txt also still produces an ordinary native Windows/Linux build from the identical source, for local testing and behavioral comparison.
- **Rendering:** librw (re3's own RenderWare-compatible renderer) targets Emscripten's OpenGL ES3/WebGL2 bindings directly; rendering code is re3's own C++ calling those bindings, unmodified.
- **Audio:** re3's existing OpenAL-based audio backend, compiled through Emscripten's built-in OpenAL implementation (itself backed by the Web Audio API).
- **Input and asset loading:** Emscripten's GLFW3 port supplies keyboard/mouse/gamepad events to re3's existing input code unmodified; a small custom JS layer bridges the browser's File-picker API and IndexedDB to Emscripten's virtual filesystem for asset and save persistence -- see §10 on copyright constraints for how assets are supplied.

4.2 Core workflow

- 1 The HTML page loads the Emscripten-generated WASM module and its JS loader, then waits for game assets before handing control to re3's own C++ entry point (`main()`).
- 2 The user is prompted to supply local data files (from their own legally owned copy of GTA III); files are read via the browser's folder picker and mounted into Emscripten's virtual filesystem, persisted to IndexedDB so future loads skip re-selecting.
- 3 The engine's main loop runs as a C++ function registered with Emscripten's `emscripten_set_main_loop`, driven by `requestAnimationFrame`, with an explicit frame-budget scheduler so the frame rate stays capped and consistent regardless of how often the browser actually invokes the callback.

- 4 Rendering, audio, input, and save/load all proceed through re3's own existing C++ systems, unmodified except for the platform-specific bindings described above.

4.3 Operational constraints for an educational, tier-2/3-hardware-friendly setting

- Target acceptable performance on mid-range consumer hardware in current Chrome and Firefox.
- Avoid dependence on novel rendering research; focus on toolchain correctness, the Emscripten porting layer, and measured performance.
- Design for deployability on plain static web hosting (e.g. GitHub Pages) -- no server-side compute or asset hosting required at runtime.

5 Solution Approach

This is an engineering porting project: the objective is a working, performant WASM build of the existing re3 C++ engine, not new rendering or AI research, and not a from-scratch reimplementation.

5.1 Engine porting

- Compile re3's existing engine systems (world state, entity logic, the librw renderer) through Emscripten largely unmodified -- changes are isolated to the small number of platform-specific files (e.g. `src/skel/glfw/`) rather than a rewrite.
- Identify and replace any assumption the engine makes that a browser genuinely cannot support (e.g. a real background POSIX thread for CD streaming), verified early via a working native build kept alongside the WASM one, rather than discovered late.
- Isolate all browser-specific behavior behind `#ifdef __EMSCRIPTEN__` blocks and a small set of exported bridge functions, so engine logic itself stays identical to the native Windows/Linux build.

5.2 Web architecture

- **Frontend:** a single static `index.html` with a canvas element, an asset-loading panel, and a live diagnostics/settings sidebar.
- **Build system:** CMake with the Emscripten toolchain (`emcmake/emcc`), producing `.wasm/.js` artifacts; no server-side runtime component is required to play.
- **Storage:** browser IndexedDB, accessed through Emscripten's IDBFS filesystem backend, persists both user-supplied game assets and save files -- in separate, isolated mounts so a save (tens of KB) never has to sync against the multi-hundred-MB asset tree.

5.3 CI/CD and fast delivery enabler

- CI builds the Emscripten target on every commit and runs a headless smoke test (module loads, reaches an initialized render state).
- CD publishes build artifacts to a static-hosting staging environment for manual playtesting.
- Automated checks track WASM output size, boot time, and a scripted frame-rate benchmark measured from real engine-side call counters, not just the on-screen display.

6 Evaluation Criterion

6.1 Primary evaluation metric

Boot success and in-browser frame rate: whether the engine reaches an interactive state, and the frame rate sustained once there.

- **Target:** reach an interactive state (main menu, then gameplay) in under 15 seconds after assets are loaded.
- **Target:** a stable, explicitly capped frame rate (tunable; verified in this build at both 50 and 55 FPS) rather than an uncontrolled rate, measured against real wall-clock time.
- **Attribution:** measured via engine-side call counters (frames actually simulated/rendered per second) exposed to JS, not the displayed counter alone, plus a boot-time timestamp from module init to first interactive frame.

6.2 Secondary evaluation metrics

- **Load time:** elapsed time from asset selection/cache-hit to playable state.
- **Output size:** compiled .wasm size relative to the native build's binary size.
- **Feature parity:** a checklist of rendering, audio, save/load, and input features verified against native re3 behavior.
- **Cross-browser compatibility:** a pass/fail matrix across Chrome, Firefox, and Safari.

6.3 Pilot validation plan

- Internal playtesting with 3-5 users who each supply assets from their own legally owned copy of GTA III.
- Compare frame rate and load-time metrics against the native build of the same engine source on matched hardware as a baseline.

7 Scalability

- 1 **Scalable (theoretically):** keep the rendering, audio, input, and storage bindings modular and decoupled from engine logic, so the port can absorb a larger asset set or higher-resolution rendering targets without a structural rewrite.
- 2 **Operationally deployable (practically):** static-hosting-only deployment, versioned and reproducible build artifacts, and a CI/CD pipeline that regenerates a fresh WASM build on every change.

8 Engine availability heuristic

A mature set of tooling and an existing engine are both already available, so the project is a porting effort, not a from-scratch build:

- **re3**, for the engine itself -- an existing, publicly available, actively maintained open-source C++ reimplementation of GTA III, used directly as the engine source rather than as a specification to reimplement.
- **Emscripten**, for compiling C/C++ to WebAssembly (with a JS glue layer).
- Emscripten's built-in browser bindings: its GLFW3 port for windowing/input, its OpenAL-compatible audio implementation targeting the Web Audio API, and IDBFS for persistent IndexedDB-backed storage.
- **CMake** with the Emscripten toolchain file, for build automation -- the same CMakeLists.txt also still drives the native Windows/Linux build.
- The native build itself, kept working alongside the WASM one throughout, as the ground-truth baseline to test and compare browser behavior against.
- Browser developer tools, for WASM debugging and profiling.

We use these mature components to focus on the Emscripten porting layer, browser bindings, and measurement, rather than inventing new infrastructure or reimplementing an engine that already exists.

9 Project Scope and Deliverables

9.1 Initial deliverable

- CMake/Emscripten build pipeline compiling re3's engine to WASM.
- Canvas boots, WebGL2 context initializes, and the engine's own startup sequence runs.
- Basic input passthrough (keyboard/gamepad) verified end-to-end.
- CI: automated Emscripten build plus a headless boot smoke test on every push.

9.2 Subsequent deliverables

- Full game loop (main menu → gameplay) running in-browser.
- A clean separation between the browser-bindings layer (rendering, audio, storage, input) and engine logic, for maintainability.
- Local asset-loading UI with IndexedDB persistence.
- Save/load support using browser storage (a dedicated IndexedDB-backed mount, kept isolated from the asset mount), and a frame-pacing tuning pass against the budget defined in §6.

10 Risks and Mitigations

- **Copyright and legal use:** GTA III's original game data remains proprietary to Take-Two/Rockstar. *Mitigation* -- re3 itself is an independent, publicly available open-source engine reimplementation, not Take-Two/Rockstar source; no copyrighted game assets or original binaries are bundled, hosted, or redistributed by this project. End users must supply data files from their own legally purchased copy of the game, loaded locally in their own browser session and never uploaded anywhere.

- **Reliance on a third-party open-source project:** re3 has itself previously been the subject of a DMCA takedown, and remains an unofficial, community-maintained reimplementation rather than an officially sanctioned one. *Mitigation* -- track re3's own upstream status, keep our porting changes isolated and clearly separated from engine logic so they can be re-applied against an updated or alternate source if needed, and be prepared to revisit specifics if a rights holder objects.
- **Emscripten/browser API gaps:** not every native API re3 assumes has a browser equivalent (e.g. real POSIX threads/semaphores, some GLFW extensions like gamepad mapping databases). *Mitigation* -- isolate each gap behind `#ifdef __EMSCRIPTEN__`, replace the affected subsystem with a browser-appropriate equivalent (e.g. CD streaming was changed from a background thread to synchronous reads, since it never needed true concurrency in a browser virtual filesystem), and gate anything requiring a user gesture (such as pointer lock) behind an explicit click.
- **Performance and frame pacing:** browsers do not guarantee `requestAnimationFrame` fires at a fixed, vsync-paced rate. *Mitigation* -- implemented a self-correcting frame scheduler in the main loop that measures real wall-clock time and paces engine work to a fixed budget regardless of how often the browser callback actually fires; verified by reading real engine-side call counters over a measured time window, not by trusting the on-screen FPS display.
- **Browser storage limits:** large asset sets may exceed IndexedDB quotas on some browsers. *Mitigation* -- support streaming/partial loads and surface clear quota warnings to the user.

11 Summary

This project ports re3 -- an existing, publicly available open-source C++ reimplementation of the Grand Theft Auto III engine -- to WebAssembly via Emscripten, so it runs directly in a browser without native installation. The engine itself is not rewritten; the work is the browser porting layer: rendering, audio, input, and persistent-storage bindings, an explicit frame-pacing scheduler, and a thin HTML/JS shell around the compiled module. It follows the course's emphasis on time-to-value through an early bootable increment, CI/CD-backed iterative delivery, and measurable evaluation criteria (boot success, frame rate, load time). It is scoped as an engineering porting effort: no original game assets are redistributed by the project, and rights-holder copyright is respected throughout -- end users provide their own legally owned data files locally.